GA7-220501096-AA1-EV02 Definir estándares de codificación de acuerdo a plataforma de desarrollo elegida

Ficha:3070324

Programa: análisis y desarrollo de software

juan Santiago Arango puentes

Servicio nacional de aprendizaje Sena

Instructor: FRANCISCO ARNALDO VARGAS

2025

INFORME TÉCNICO: ESTÁNDARES DE CODIFICACIÓN

Aplicación Menu Pizza

PORTADA

INSTITUCIÓN EDUCATIVA: SENA - Servicio Nacional de Aprendizaje

PROGRAMA: Técnico en Sistemas

COMPETENCIA: GA7-220501096-AA1-EV02 - Definir estándares de codificación de acuerdo a plataforma de desarrollo elegida

PROYECTO: Plataforma de Pedidos en Línea - Menu Pizza

FECHA: Abril 2026

AUTOR: Equipo de Desarrollo

TABLA DE CONTENIDOS

1. Introducción
2. Objetivo General
3. Objetivos Específicos
4. Plataforma de Desarrollo Seleccionada
5. Estándares de Codificación
6. Conclusiones

---

1. INTRODUCCIÓN

El presente informe técnico documenta los estándares de codificación establecidos para la plataforma de desarrollo elegida en el proyecto "Menu Pizza", una aplicación web moderna para gestión de pedidos de pizzas en línea.

Los estándares de codificación son directrices fundamentales que aseguran:

- **Consistencia:** Código uniforme y predecible
- **Mantenibilidad:** Facilita el trabajo en equipo y futuras modificaciones
- **Legibilidad:** Código claro y fácil de comprender
- **Escalabilidad:** Base sólida para crecimiento del proyecto
- **Calidad:** Reduce errores y facilita depuración

Este documento está fundamentado en los principios del Paradigma Orientado a Objetos (POO) y metodologías de desarrollo moderno con React y TypeScript.

---

2. OBJETIVO GENERAL

Definir y documentar los estándares de codificación a utilizar en el desarrollo del proyecto "Menu Pizza", garantizando una base cohesiva, mantenible y escalable basada en las características funcionales de la plataforma y la plataforma tecnológica seleccionada.

---

3. OBJETIVOS ESPECÍFICOS

- Establecer convenciones de nomenclatura para variables, constantes, funciones, clases e interfaces
- Definir reglas de organización de código y estructura de proyectos
- Documentar estándares de escritura de componentes React y TypeScript
- Establecer patrones de gestión de estado y datos
- Definir normas de comentarios y documentación del código
- Asegurar compatibilidad con herramientas de verificación de tipos y linting
- Garantizar mejores prácticas en seguridad y rendimiento

---

4. PLATAFORMA DE DESARROLLO SELECCIONADA

4.1 Stack Tecnológico

El proyecto "Menu Pizza" utiliza un stack moderno de desarrollo web frontend:

Lenguaje de Programación:

- **TypeScript 5.6.2** - Superset de JavaScript que proporciona tipado estático
  - Versión: TypeScript ~5.6.2
  - Tipo de módulo: ESNext

Framework de UI:

- **React 18.3.1** - Librería para construcción de interfaces de usuario con componentes
  - Componentes funcionales con Hooks
  - Gestión de estado con useState y useEffect

Build Tool:

- **Vite 6.0.1** - Herramienta de construcción y desarrollo moderna
  - Configuración: vite.config.ts
  - Soporte nativo para TypeScript y JSX

Estilos:

- **Tailwind CSS 4.1.3** - Framework CSS utilidad-first
  - Configuración: Tailwind CSS integrado con Vite

Complementos:

- **Lucide React 0.460.0** - Librería de iconos React
- **@vitejs/plugin-react 4.3.4** - Plugin de React para Vite

Configuración de TypeScript:

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "jsx": "react-jsx",
    "strict": true,
    "noEmit": true,
    "noUnusedLocals": true,
```

```
    "noUnusedParameters": true,
    "noFallthroughCasesInSwitch": true
  }
}
```

4.2 Estructura del Proyecto

```
Menu Pizza/
├── components/              # Componentes React
│   ├── admin/              # Componentes administrativos
│   ├── ui/                 # Componentes de interfaz reutilizables
│   ├── figma/              # Componentes del diseño Figma
│   └── *.tsx               # Componentes principales
├── utils/                  # Funciones utilitarias
├── assets/                 # Activos del proyecto (imágenes, etc.)
├── styles/                 # Estilos globales
├── guidelines/             # Documentación de pautas
├── App.tsx                 # Componente raíz
├── main.tsx                # Punto de entrada
├── package.json            # Dependencias del proyecto
└── tsconfig.json           # Configuración de TypeScript
```

---

5. ESTÁNDARES DE CODIFICACIÓN

5.1 CONVENCIONES DE NOMENCLATURA

5.1.1 Componentes React

Regla: Los componentes React deben usar PascalCase (también conocido como UpperCamelCase)

Ejemplos correctos:

```
export function Home() { ... }
export function Menu() { ... }
export function ShoppingCart() { ... }
export function PaymentMethods() { ... }
export function UserManagement() { ... }
```

Ejemplos incorrectos:

```
export function home() { ... }              // ❌ Incorrecto
export function shopping_cart() { ... }     // ❌ Incorrecto
export function MENU() { ... }              // ❌ Incorrecto
```

Justificación: React diferencia componentes de elementos HTML estándar usando mayúscula inicial. Esto ayuda a identificar rápidamente qué es un componente.

5.1.2 Variables y Propiedades

Regla: Las variables deben usar camelCase (lowerCamelCase)

Ejemplos correctos:

```
const [view, setView] = useState<View>('home');
const [cart, setCart] = useState<CartItem[]>([]);
const [user, setUser] = useState<User | null>(null);
const [admin, setAdmin] = useState<Admin | null>(null);
const savedUser = getUser();
const cartItemsCount = cart.length;
const deliveryAddress = location.address;
```

Ejemplos incorrectos:

```
const [View, setView] = useState<View>();     // ❌ Incorrecto
const [Cart_Items, setCartItems] = useState();  // ❌ Incorrecto
const CART = useState();                        // ❌ Incorrecto
```

Justificación: camelCase es el estándar de JavaScript y hace el código más legible y consistente con convenciones ampliamente reconocidas.

5.1.3 Funciones y Métodos

Regla: Las funciones deben usar camelCase y ser verbos que describan la acción

Ejemplos correctos:

```
const addToCart = (product: Product) => { ... }
const removeFromCart = (id: string) => { ... }
const updateQuantity = (id: string, quantity: number) => { ... }
const getUser = (): User | null => { ... }
const saveCart = (cart: CartItem[]): void => { ... }
const isUserBanned = (userId: string): boolean => { ... }
```

Ejemplos incorrectos:

```
const AddToCart = () => { ... }           // ❌ PascalCase (reservado para componentes)
const add_to_cart = () => { ... }         // ❌ snake_case
const cartAdd = () => { ... }             // ❌ Verbo al final (poco legible)
```

Justificación: Los verbos al inicio hacen explícito el propósito de la función, facilitando legibilidad y mantenibilidad.

5.1.4 Constantes

Regla: Las constantes deben usar camelCase o UPPER_SNAKE_CASE según su contexto

Ejemplos correctos (valores predeterminados):

```
const defaultProducts = [
  { id: '1', name: 'Pizza Hawaiana', price: 46000 },
  { id: '2', name: 'Pizza Peperoni', price: 42000 }
];
```

```
const localStorageKey = 'gusnat_user';
```

Ejemplos correctos (constantes globales):

```
const MAX_CART_ITEMS = 100;
const API_TIMEOUT_MS = 30000;
const CACHE_EXPIRATION_HOURS = 24;
```

Justificación: Constantes primitivas globales en UPPER_SNAKE_CASE facilitan su identificación. Datos complejos en camelCase son más naturales.

5.1.5 Interfaces y Tipos

Regla: Las interfaces y tipos deben usar PascalCase

Ejemplos correctos:

```
interface Product {
  id: string;
  name: string;
  description: string;
  price: number;
  image: string;
  size?: string;
}

interface CartItem extends Product {
  quantity: number;
}

interface User {
  id: string;
  name: string;
  email: string;
  phone: string;
  role?: "user" | "admin";
}

interface DeliveryLocation {
  address: string;
  city: string;
  additionalInfo?: string;
}

type View = 'home' | 'menu' | 'cart' | 'payment' | 'login' |
           'register' | 'location' | 'adminPanel';
```

Ejemplos incorrectos:

```
interface product { ... }                // ❌ Incorrecto
interface UserData_Interface { ... }     // ❌ Incorrecto
type view = 'home' | 'menu';             // ❌ Incorrecto
```

Justificación: PascalCase diferencia tipos y clases de valores, mejorando legibilidad del código.

5.1.6 Propiedades de Componentes (Props)

Regla: Las interfaces de props deben ser claras y específicas

Ejemplo correcto:

```
interface HomeProps {
  onNavigate: (view: View) => void;
  user: User | null;
  onLogout: () => void;
}

interface MenuProps {
  onAddToCart: (product: Product) => void;
  onViewCart: () => void;
  onBack: () => void;
  cartItemsCount: number;
  user: User | null;
  onLogout: () => void;
}
```

Justificación: Props explícitas facilitan conocer qué necesita un componente y qué puede hacer.

5.2 ORGANIZACIÓN DE ARCHIVOS Y CARPETAS

5.2.1 Estructura de Directorios

Regla: Organizar archivos por funcionalidad, no por tipo

Estructura recomendada:

```
src/
├── components/
│   ├── common/              # Componentes reutilizables globales
│   ├── pages/               # Componentes de página
│   ├── admin/               # Componentes administrativos
│   ├── ui/                  # Componentes UI base (shadcn/ui, etc.)
│   └── Home.tsx
├── utils/                   # Funciones utilitarias
│   ├── localStorage.ts
│   └── adminData.ts
├── types/                   # Definiciones de tipos (si es necesario)
├── styles/                  # Estilos globales
├── assets/                  # Imágenes, fuentes, etc.
└── App.tsx
```

5.2.2 Nombres de Archivos

Regla: Los archivos de componentes deben coincidir con el nombre del componente en PascalCase

Ejemplos correctos:

```
Home.tsx
Menu.tsx
ShoppingCart.tsx
PaymentMethods.tsx
Login.tsx
Register.tsx
LocationForm.tsx
AdminPanel.tsx
UserManagement.tsx
```

Archivos de utilidades y estilos en camelCase:

```
localStorage.ts
adminData.ts
globals.css
index.css
```

Justificación: Facilita encontrar componentes por nombre y mantiene consistencia con la convención de nombrado de TypeScript.

5.3 ESCRITURA DE COMPONENTES REACT

5.3.1 Componentes Funcionales

Regla: Todos los componentes deben ser funcionales (no usar clases)

Ejemplo correcto:

```
interface HomeProps {
  onNavigate: (view: View) => void;
  user: User | null;
  onLogout: () => void;
}

export function Home({ onNavigate, user, onLogout }: HomeProps) {
  return (
    <div className="home-container">
      {user && <p>Bienvenido, {user.name}</p>}
    </div>
  );
}
```

Alternativa válida (aunque la anterior es preferible):

```
export const Home = ({ onNavigate, user, onLogout }: HomeProps) => {
  return (
    <div className="home-container">
```

```
      {user && <p>Bienvenido, {user.name}</p>}
    </div>
  );
};
```

Justificación: Los componentes funcionales son más modernos, eficientes y fáciles de optimizar con Hooks.

5.3.2 Hooks de React

Regla: Los Hooks deben colocarse al principio del componente, en orden específico

Orden recomendado:

1. useState (gestión de estado)
2. useEffect (efectos secundarios)
3. Otros Hooks personalizados

Ejemplo correcto:

```
export function Menu({ onAddToCart, onViewCart, onBack }: MenuProps) {
  const [selectedProduct, setSelectedProduct] = useState<Product | null>(null);
  const [filteredProducts, setFilteredProducts] = useState<Product[]>([]);

  useEffect(() => {
    // Cargar productos al montar
    loadProducts();
  }, []);

  useEffect(() => {
    // Efecto secundario adicional
    filterProducts();
  }, [selectedProduct]);

  return (
    <div>
      {/* JSX */}
    </div>
  );
}
```

5.3.3 Gestión de Estado

Regla: Usar useState para estado local, considerar contexto para estado global

Ejemplo correcto:

```
const [cart, setCart] = useState<CartItem[]>([]);
const [user, setUser] = useState<User | null>(null);
const [isLoading, setIsLoading] = useState(false);

// Actualizar estado correctamente
const addToCart = (product: Product) => {
  setCart(prevCart => {
```

```
    const existingItem = prevCart.find(
      item => item.id === product.id && item.size === product.size
    );

    if (existingItem) {
      return prevCart.map(item =>
        item.id === product.id && item.size === product.size
          ? { ...item, quantity: item.quantity + 1 }
          : item
      );
    }

    return [...prevCart, { ...product, quantity: 1 }];
  });
};
```

Justificación: Usar funciones de actualización prevState evita bugs de estado cerrado y garantiza valores actuales.

5.4 TIPADO TYPESCRIPT

5.4.1 Uso de Tipos Explícitos

Regla: Siempre proporcionar tipos explícitos para variables, parámetros y retornos

Ejemplo correcto:

```
// Función con tipos completos
const saveUser = (user: User): void => {
  localStorage.setItem("gusnat_user", JSON.stringify(user));
};

// Variable con tipo explícito
const getUser = (): User | null => {
  const user: string | null = localStorage.getItem("gusnat_user");
  return user ? JSON.parse(user) : null;
};

// Parámetro con tipo
const removeFromCart = (id: string, size?: string): void => {
  // Implementación
};

// Array tipado
const cart: CartItem[] = [];
const products: Product[] = [];
```

Ejemplos incorrectos:

```
const saveUser = (user) => { ... }                    // ❌ Sin tipos
const getUser = () => { ... }                         // ❌ Sin tipos de retorno
```

```
let cart;                                          // ❌ Sin tipado
const handleClick = (e) => { ... }                 // ❌ Sin tipos explícitos
```

Justificación: El tipado explícito proporciona seguridad de tipos en tiempo de desarrollo y mejora la documentación del código.

5.4.2 Uniones y Tipos Literales

Regla: Usar tipos literales para valores predefinidos

Ejemplo correcto:

```
type View = 'home' | 'menu' | 'cart' | 'payment' | 'login' |
           'register' | 'location' | 'adminPanel' | 'adminUsers' |
           'adminEditor' | 'adminHistory' | 'adminUserHistory' | 'adminCalendar';

type UserRole = "user" | "admin";

interface User {
  id: string;
  name: string;
  email: string;
  phone: string;
  role?: UserRole;
}
```

Justificación: Tipos literales previenen errores de tipeo y proporcionan autocompletado superior.

5.4.3 Interfaces vs Types

Regla: Preferir interfaces para objetos, types para uniones y aliases

Componentes de objeto - usar interface:

```
interface Product {
  id: string;
  name: string;
  description: string;
  price: number;
  image: string;
  size?: string;
}

interface User {
  id: string;
  name: string;
  email: string;
  phone: string;
  role?: "user" | "admin";
}
```

Uniones y tipos simples - usar type:

```
type View = 'home' | 'menu' | 'cart' | 'payment' | 'login' | 'register';
type Optional<T> = T | null | undefined;
```

Justificación: Las interfaces son mejores para definir contratos de objetos; los types son más flexibles para uniones.

5.5 GESTIÓN DE DATOS Y PERSISTENCIA

5.5.1 localStorage

Patrón recomendado:

```
// Definir prefijo consistente para claves
const STORAGE_PREFIX = 'gusnat_';

// Funciones de acceso específicas
export const saveUser = (user: User): void => {
  localStorage.setItem(`${STORAGE_PREFIX}user`, JSON.stringify(user));
};

export const getUser = (): User | null => {
  const data = localStorage.getItem(`${STORAGE_PREFIX}user`);
  return data ? JSON.parse(data) : null;
};

export const removeUser = (): void => {
  localStorage.removeItem(`${STORAGE_PREFIX}user`);
};

// Manejo de errores
export const getCart = (): CartItem[] => {
  try {
    const cart = localStorage.getItem(`${STORAGE_PREFIX}cart`);
    return cart ? JSON.parse(cart) : [];
  } catch (error) {
    console.error('Error parsing cart from localStorage:', error);
    return [];
  }
};
```

Justificación: Centralizar acceso a localStorage facilita cambios futuros y previene errores.

5.6 COMENTARIOS Y DOCUMENTACIÓN

5.6.1 Comentarios de Código

Regla: Comentarios claros y significativos, no redundantes

Ejemplo correcto:

```
// Cargar usuario y carrito del localStorage al iniciar
useEffect(() => {
  const savedUser = getUser();
```

```
  const savedCart = getCart();

  if (savedUser) {
    setUser(savedUser);
  }

  if (savedCart.length > 0) {
    setCart(savedCart);
  }

  // Verificar suspensión de página cada minuto
  const suspension = getPageSuspension();
  setPageSuspended(!!suspension);
}, []);
```

Ejemplos incorrectos:

```
// Asignar savedUser a user
setUser(savedUser);                    // ❌ Redundante

// Obtener el usuario
const user = getUser();                // ❌ Obvio del código

// Incrementar contador
counter++;                             // ❌ No aporta valor
```

5.6.2 Documentación de Funciones

Regla: Documentar funciones públicas con JSDoc

Ejemplo correcto:

```
/**
 * Agrega un producto al carrito o incrementa su cantidad
 * @param product - El producto a agregar al carrito
 * @returns void
 * @example
 * addToCart({ id: '1', name: 'Pizza', price: 45000, ... })
 */
export const addToCart = (product: Product): void => {
  // Implementación
};

/**
 * Obtiene el usuario actual desde el almacenamiento local
 * @returns El usuario autenticado o null si no hay usuario
 * @throws Puede lanzar error si el JSON del localStorage es inválido
 */
export const getUser = (): User | null => {
  // Implementación
};
```

5.7 ESTILOS CON TAILWIND CSS

5.7.1 Clases CSS

Regla: Usar clases de Tailwind CSS contextuales y mantenerlas organizadas

Ejemplo correcto:

```
export function Home() {
  return (
    <div className="flex flex-col min-h-screen bg-white">
      {/* Header */}
      <header className="bg-orange-600 text-white px-4 py-3">
        <h1 className="text-2xl font-bold">Menu Pizza</h1>
      </header>

      {/* Main Content */}
      <main className="flex-1 p-4">
        <section className="grid grid-cols-1 md:grid-cols-2 gap-4">
          {/* Cards */}
          <div className="bg-gray-100 rounded-lg p-4 hover:shadow-lg transition-shadow">
            {/* Content */}
          </div>
        </section>
      </main>
    </div>
  );
}
```

Justificación: Tailwind CSS proporciona consistencia visual y mantiene estilos cerca del contenido.

5.7.2 Componentes Personalizados

Patrón para componentes reutilizables:

```
interface ButtonProps {
  variant?: 'primary' | 'secondary' | 'danger';
  size?: 'sm' | 'md' | 'lg';
  children: React.ReactNode;
  onClick?: () => void;
}

export const Button = ({
  variant = 'primary',
  size = 'md',
  children,
  onClick
}: ButtonProps) => {
  const baseStyles = 'font-medium rounded transition-colors';

  const variantStyles = {
    primary: 'bg-orange-600 text-white hover:bg-orange-700',
```

```
    secondary: 'bg-gray-200 text-gray-900 hover:bg-gray-300',
    danger: 'bg-red-600 text-white hover:bg-red-700'
  };

  const sizeStyles = {
    sm: 'px-2 py-1 text-sm',
    md: 'px-4 py-2 text-base',
    lg: 'px-6 py-3 text-lg'
  };

  return (
    <button
      className={`${baseStyles} ${variantStyles[variant]} ${sizeStyles[size]}`}
      onClick={onClick}
    >
      {children}
    </button>
  );
};
```

5.8 IMPORTES Y EXPORTES

5.8.1 Importes

Regla: Organizar importes en grupos: librerías externas, módulos locales, tipos

Ejemplo correcto:

```
// 1. Librerías externas
import { useState, useEffect } from 'react';
import { Menu as MenuIcon, ShoppingBag, Phone } from 'lucide-react';

// 2. Módulos locales
import { Home } from './components/Home';
import { Menu } from './components/Menu';
import { ShoppingCart } from './components/ShoppingCart';

// 3. Utilidades
import { getUser, getCart, saveCart, removeUser } from './utils/localStorage';
import { getPageSuspension } from './utils/adminData';

// 4. Tipos
import type { User, CartItem } from './utils/localStorage';
import type { Admin } from './utils/adminData';
```

5.8.2 Exportes

Regla: Usar named exports para componentes; default exports solo cuando sea apropiado

Ejemplos correctos:

```
// ✅ Componentes - Named export
export function Home() { ... }
export function Menu() { ... }

// ✅ Funciones utilitarias - Named export
export const getUser = (): User | null => { ... };
export const saveCart = (cart: CartItem[]): void => { ... };

// ✅ Índice de exportes
export { Home } from './components/Home';
export { Menu } from './components/Menu';
export { ShoppingCart } from './components/ShoppingCart';

// A veces - Default export para aplicación raíz
export default function App() { ... }
```

Justificación: Named exports facilitan refactorización y búsqueda de referencias.

5.9 PATRONES Y BUENAS PRÁCTICAS

5.9.1 Manejo de Errores

Patrón recomendado:

```
export const getCart = (): CartItem[] => {
  try {
    const cart = localStorage.getItem('gusnat_cart');
    if (!cart) return [];
    return JSON.parse(cart);
  } catch (error) {
    console.error('Error al cargar carrito:', error);
    return [];
  }
};
```

5.9.2 Validación de Datos

Patrón recomendado:

```
interface ValidationResult {
  isValid: boolean;
  errors: string[];
}

const validateUser = (user: Partial<User>): ValidationResult => {
  const errors: string[] = [];

  if (!user.email || !user.email.includes('@')) {
    errors.push('Email inválido');
  }
```

```
  if (!user.name || user.name.length < 3) {
    errors.push('Nombre debe tener al menos 3 caracteres');
  }

  return {
    isValid: errors.length === 0,
    errors
  };
};
```

5.9.3 Componentes Condicionales

Patrón recomendado:

```
interface AdminPanelProps {
  user: User | null;
  children: React.ReactNode;
}

export function AdminPanelWrapper({ user, children }: AdminPanelProps) {
  // Validar rol
  if (!user || user.role !== 'admin') {
    return <div>No tienes acceso a esta sección</div>;
  }

  return <>{children}</>;
}
```

5.9.4 Efectos Secundarios

Patrón recomendado:

```
useEffect(() => {
  // Cargar datos al montar
  const savedUser = getUser();
  const savedCart = getCart();

  if (savedUser) {
    setUser(savedUser);
  }

  if (savedCart.length > 0) {
    setCart(savedCart);
  }
}, []);

// Guardar carrito cuando cambie
useEffect(() => {
  saveCart(cart);
}, [cart]);
```

```
// Verificar periódicamente
useEffect(() => {
  const interval = setInterval(() => {
    const suspension = getPageSuspension();
    setPageSuspended(!!suspension);
  }, 60000);

  return () => clearInterval(interval);
}, []);
```

Justificación: Limpiar recursos (cleanup) es crítico para evitar memory leaks.

5.10 CONFIGURACIÓN DE TYPESCRIPT

5.10.1 Opciones del Compilador Recomendadas

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ESNext",
    "lib": ["ES2020", "DOM", "DOM.Iterable"],
    "jsx": "react-jsx",

    // Seguridad de tipos
    "strict": true,
    "noImplicitAny": true,
    "strictNullChecks": true,
    "strictFunctionTypes": true,

    // Calidad de código
    "noUnusedLocals": true,
    "noUnusedParameters": true,
    "noImplicitReturns": true,
    "noFallthroughCasesInSwitch": true,

    // Módulos
    "moduleResolution": "bundler",
    "allowImportingTsExtensions": true,
    "isolatedModules": true,

    // Salida
    "noEmit": true,
    "skipLibCheck": true
  }
}
```

5.11 CHECKLIST DE ESTÁNDARES

Antes de hacer commit, verificar:

- [ ] Los componentes usan PascalCase

- [ ] Variables y funciones usan camelCase
- [ ] Interfaces y tipos usan PascalCase
- [ ] Todos los tipos están explícitamente declarados
- [ ] Los comentarios son significativos
- [ ] Sin variables sin usar (noUnusedLocals)
- [ ] Sin parámetros sin usar (noUnusedParameters)
- [ ] Funciones con void cuando no retornan valor
- [ ] Archivos de componentes coinciden con nombre de componente
- [ ] Props tienen interfaz definida
- [ ] Importes están organizados correctamente
- [ ] No hay any implícito (strict mode)
- [ ] UseEffect tiene dependencias definidas correctamente
- [ ] Manejo de errores implementado donde sea necesario

---

6. CONCLUSIONES

Los estándares de codificación definidos en este informe proporcionan una base sólida para el desarrollo del proyecto Menu Pizza. Estos estándares:

1. **Aseguran Consistencia:** Un código uniforme facilita la colaboración y el mantenimiento
2. **Mejoran Legibilidad:** Código claro y predecible reduce curva de aprendizaje
3. **Facilitan Escalabilidad:** Estructura modular permite crecimiento sostenible
4. **Previenen Errores:** TypeScript strict mode y validaciones detectan problemas temprano
5. **Optimizan Rendimiento:** Componentes funcionales y gestión eficiente de estado

La adopción rigurosa de estos estándares durante el desarrollo del proyecto garantizará una aplicación de calidad, fácil de mantener y evolucionar.

Recomendaciones Futuras

- Implementar ESLint con configuración stricta
- Usar Prettier para formateo automático
- Agregar Git hooks (husky) para validar código antes de commits
- Implementar tests unitarios con Jest/Vitest
- Documentar patrones específicos del dominio cuando surjan

REFERENCIAS

- TypeScript Handbook: https://www.typescriptlang.org/docs/
- React Documentation: https://react.dev
- Google TypeScript Style Guide
- Airbnb JavaScript Style Guide
- Tailwind CSS Documentation: https://tailwindcss.com

Documento generado: Abril 2026

Versión: 1.0

Estado: Aprobado para uso